# AgroFácil Visão: detecção em tempo real de doenças em folhas para o pequeno produtor

**Applied Computer Vision · Global Solution 2026.2 · Desafio CitizenTech**

Engenharia de Software · 4º ano (4ES) · FIAP

Professores: Arnaldo Viana e Paulo Rodrigues

## Integrantes


| Nome completo | RM |
|---|---|
| Julia Ortiz | RM550204 |
| Julia Palomari | RM551910 |
| Leticia Baptista | RM550289 |
| Vinicius Sobreira Borges | RM97767 |


Repositório: https://github.com/viniciusborgess/agrofacil

Vídeo de demonstração: (link do vídeo)

## Resumo

Pequenos produtores rurais costumam perceber doenças nas folhas tarde, muitas vezes sem assistência técnica por perto, o que atrasa o manejo e aumenta perdas. Este trabalho apresenta o AgroFácil Visão, componente de visão computacional da plataforma AgroFácil, que detecta e localiza em tempo real folhas sadias e doentes de oito culturas, a partir da câmera do celular ou de uma foto. Treinamos de forma customizada um detector YOLO11n, pré-treinado no COCO, com o dataset aberto PlantDoc, restrito a 20 classes. Antes do treino, auditamos os dados: removemos 59 duplicatas, inclusive 13 casos de vazamento entre treino e teste, e descartamos anotações incompatíveis. O limiar de decisão foi calibrado apenas na validação, e o teste foi usado uma única vez. No conjunto de teste, com 168 imagens, o modelo obteve mAP@50 de 0,617 e mAP@50-95 de 0,479. Na decisão entregue ao produtor, sinalizou 89,6% das folhas doentes e chamou de saudáveis 4,0% delas. Exportado para ONNX, processa cerca de 27 imagens por segundo em CPU, sem placa de vídeo. Também medimos o comportamento fora do cenário ideal, com fotos degradadas, culturas desconhecidas e imagens sem planta. A solução é servida por uma API que não armazena imagens, devolve orientações em linguagem simples e recomenda confirmação humana. Os resultados mostram viabilidade como ferramenta de triagem, não de diagnóstico, sobretudo porque os dados não vêm de lavouras brasileiras e quase não contêm pragas. O código, os dados de avaliação e as instruções para reproduzir tudo estão no repositório público do projeto.

248 palavras.


## 1. Introdução

O AgroFácil é a CitizenTech da equipe: uma plataforma que ajuda pequenos produtores rurais a decidir sobre irrigação, plantio e cuidados com a lavoura usando dados climáticos e inteligência artificial. Uma das funcionalidades previstas é identificar problemas na plantação por imagem. Este relatório documenta esse componente, chamado AgroFácil Visão.

O produtor costuma notar uma mancha na folha antes de ter acesso a quem saiba interpretá-la. Algumas doenças, como a requeima do tomate e da batata, avançam em poucos dias em clima frio e úmido, e a demora no primeiro cuidado pesa no resultado da safra. Um celular comum, apontado para a folha, pode reduzir essa espera: o sistema indica onde está o possível problema, sugere o primeiro manejo em linguagem simples e recomenda confirmar com um agrônomo ou com a assistência técnica (ATER).

A disciplina pede detecção de objetos em tempo real com treinamento customizado. Escolhemos detecção, e não apenas classificação da imagem inteira, porque uma foto de campo mostra várias folhas, fundo e solo: a caixa mostra ao produtor qual folha motivou o alerta, o que torna o resultado verificável.

## 2. Objetivo

Treinar de forma customizada um detector de objetos YOLO11n que localize e classifique, em tempo real e em CPU comum, folhas sadias e doentes de oito culturas da agricultura familiar, integrá-lo à plataforma AgroFácil por meio de uma API segura e medir com rigor seu desempenho e seus limites.

49 palavras.

## 3. Explicação técnica das ferramentas

| Ferramenta | Papel no projeto | Por que foi escolhida |
|---|---|---|
| PlantDoc (CC BY 4.0) | Dataset de 2.598 fotos reais de folhas, anotadas com caixas, em 13 espécies. | É um dos poucos datasets abertos de detecção com fotos fora do laboratório. |
| YOLO11n (Ultralytics) | Arquitetura de detecção de um estágio: backbone convolucional, pescoço com fusão de escalas e cabeça sem âncoras, que prevê caixa e classe para cada célula. | É o menor modelo da família (2,6 milhões de parâmetros), adequado a CPU e celulares de entrada. |
| Transfer learning (COCO) | Pesos pré-treinados em 80 classes gerais, ajustados para as 20 classes do projeto. | Com cerca de 1.400 imagens de treino, partir do zero não convergiria bem. |
| PyTorch + CUDA (Google Colab) | Treino em uma GPU NVIDIA T4 gratuita do Google Colab. | No notebook da equipe (Apple M1, 8 GB) cada época levava cerca de 17 minutos; na T4, as 40 épocas levaram cerca de 13. |
| ONNX + ONNX Runtime | Modelo exportado e executado em CPU, com pré e pós-processamento (letterbox e NMS) próprios em NumPy. | Imagem de produção sem PyTorch, menor e mais barata de hospedar. |
| FastAPI | API com POST /v1/detections, que devolve JSON com caixas, rótulos e orientação. | Contrato claro para a plataforma AgroFácil e documentação automática. |
| Navegador (getUserMedia) | Demonstração com a câmera: envia quadros de 480 px e desenha as caixas. | Funciona no celular sem instalar aplicativo. |
| imagehash (pHash) | Encontra fotos quase idênticas entre treino e teste. | Evita resultados inflados por vazamento. |
| Docker | Empacota a API com usuário sem privilégios. | Integra-se ao componente de DevOps do grupo. |

## 4. Metodologia

1. Escopo. Mantivemos 20 classes de oito culturas comuns em pequenas propriedades: tomate, milho, batata, pimentão, abóbora, soja, morango e uva. Ficaram de fora as fruteiras de clima temperado e duas classes sem exemplos suficientes; entre elas, a única praga do dataset (ácaro-rajado), com apenas 2 imagens.
2. Extração. Lemos os arquivos direto dos objetos do git, porque o repositório original tem nomes que só diferem por maiúsculas e se sobrescrevem no macOS e no Windows.
3. Validação das anotações. Pareamos imagem e XML, descartamos 22 anotações cujas dimensões não batiam com a imagem e caixas degeneradas, e corrigimos a orientação EXIF.

4 Deduplicação. Agrupamos duplicatas exatas e quase idênticas (distância de pHash até 4). Removemos 59 imagens, inclusive 13 grupos presentes ao mesmo tempo no treino e no teste. Nesses casos, preservamos a cópia do teste oficial.

5 Divisão. Do treino oficial, separamos 15% para validação, estratificado pela classe principal (semente 42). Resultado: 1392 imagens de treino, 246 de validação e 168 de teste, este intocado até a avaliação final.

6 Treino. YOLO11n pré-treinado no COCO, imagens de 480 px, lote 16, até 40 épocas com taxa de aprendizado cosseno e parada antecipada (paciência 12), em uma GPU NVIDIA T4 do Google Colab. Foram 40 épocas. Além do mosaico padrão, usamos rotação de 15° e inversão vertical, porque folhas não têm orientação fixa.

7 Calibração. Escolhemos na validação o limiar de confiança que maximiza o F1 médio: 0,33. Abaixo dele o sistema não afirma nada e pede outra foto.

8 Avaliação. No teste, medimos mAP, precisão e recall por classe; a decisão final por imagem, com intervalos de confiança por bootstrap; a robustez a fotos degradadas; respostas para culturas desconhecidas e imagens sem planta; e a latência em CPU.

9 Integração. Exportamos para ONNX, servimos por FastAPI e conferimos que a saída em ONNX é idêntica à do PyTorch para a mesma entrada (25 de 25 imagens, diferença zero).

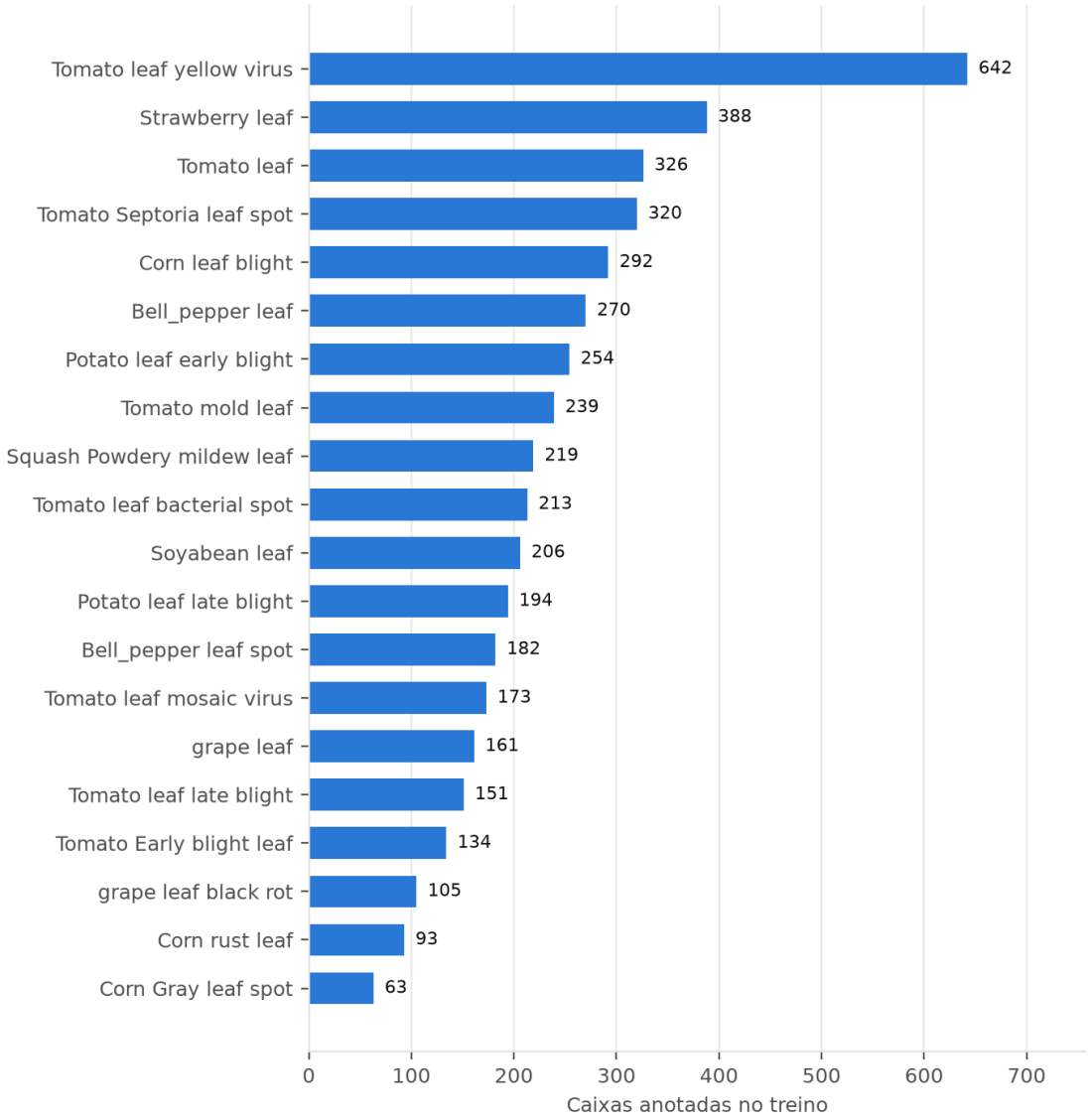



Figura 1. Caixas anotadas por classe no treino (4625 no total). O desbalanceamento é alto: as classes com menos exemplos tendem a ter desempenho pior.

# 5. Resultados

## 5.1 Métricas de detecção

| Conjunto | mAP@50 | mAP@50-95 | Precisão | Recall |
|---|---|---|---|---|
| Validação | 0,556 | 0,395 | - | - |
| Teste (168 imagens) | 0,617 | 0,479 | 0,527 | 0,610 |

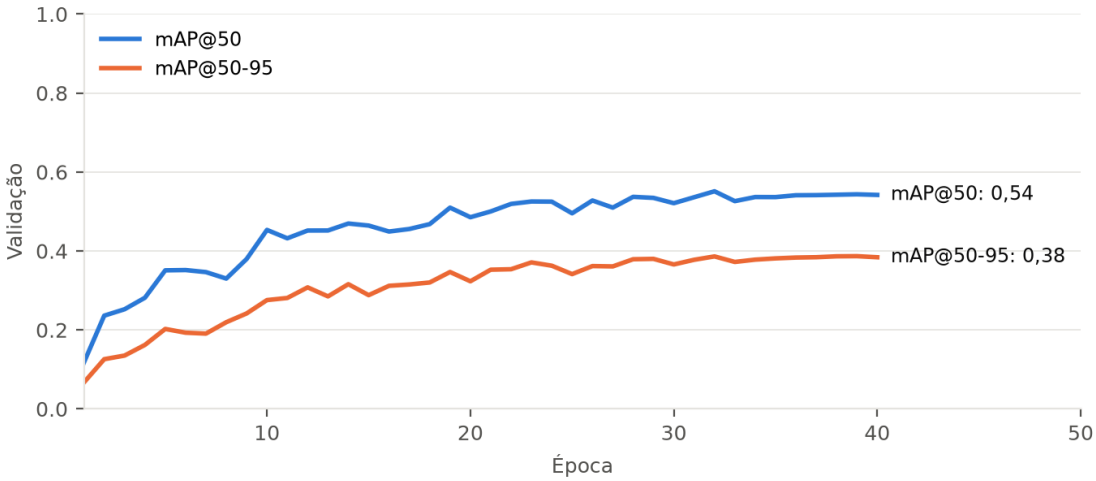



Figura 2. Evolução do mAP na validação durante o treino.

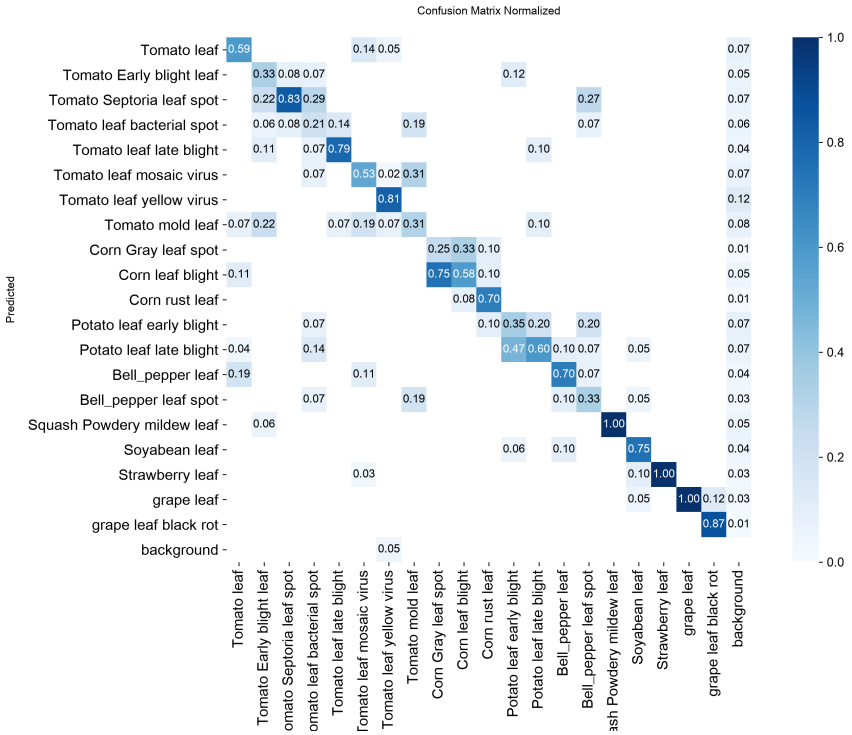



Figura 3. Matriz de confusão normalizada no teste. As confusões mais frequentes são entre doenças da mesma cultura, cujos sintomas se parecem.

## 5.2 Desempenho por classe (teste)

| Classe | Precisão | Recall | mAP@50 | mAP@50-95 |
|---|---|---|---|---|
| Strawberry leaf | 0,84 | 0,97 | 0,99 | 0,83 |
| Corn rust leaf | 0,88 | 0,75 | 0,96 | 0,81 |
| grape leaf | 0,71 | 0,87 | 0,92 | 0,77 |
| Soyabean leaf | 0,67 | 0,80 | 0,88 | 0,76 |
| grape leaf black rot | 0,71 | 0,88 | 0,87 | 0,72 |
| Squash Powdery mildew leaf | 0,37 | 0,83 | 0,84 | 0,68 |
| Corn leaf blight | 0,63 | 0,71 | 0,76 | 0,66 |
| Tomato leaf late blight | 0,69 | 0,64 | 0,75 | 0,62 |
| Tomato leaf yellow virus | 0,55 | 0,69 | 0,65 | 0,35 |
| Tomato leaf mosaic virus | 0,87 | 0,36 | 0,65 | 0,41 |
| Tomato Septoria leaf spot | 0,49 | 0,79 | 0,57 | 0,35 |
| Potato leaf early blight | 0,51 | 0,41 | 0,51 | 0,42 |
| Potato leaf late blight | 0,33 | 0,70 | 0,48 | 0,36 |
| Tomato leaf | 0,38 | 0,41 | 0,47 | 0,32 |
| Tomato Early blight leaf | 0,51 | 0,28 | 0,44 | 0,27 |
| Bell_pepper leaf | 0,31 | 0,76 | 0,41 | 0,31 |
| Bell_pepper leaf spot | 0,44 | 0,33 | 0,35 | 0,29 |
| Tomato leaf bacterial spot | 0,20 | 0,21 | 0,31 | 0,24 |
| Corn Gray leaf spot | 0,27 | 0,48 | 0,27 | 0,22 |
| Tomato mold leaf | 0,16 | 0,31 | 0,25 | 0,18 |

Classes com poucas imagens no teste têm métricas muito instáveis; leia a tabela como indicação.

## 5.3 A decisão que o produtor recebe

O mAP mede caixas. O produtor, porém, recebe uma decisão por foto: possível doença, folha saudável ou pedido de nova foto. Medimos essa decisão no teste, rodando exatamente o código da API.

| Indicador | Resultado |
|---|---|
| Folhas doentes sinalizadas como possível doença | 89,6% (IC95% 84,0% a 94,4%; n=125) |
| Folhas doentes ditas saudáveis (erro mais grave) | 4,0% (IC95% 0,8% a 8,0%; n=125) |
| Folhas sadias reconhecidas como saudáveis | 81,4% (IC95% 69,8% a 93,0%; n=43) |
| Folhas sadias com alarme falso de doença | 11,6% (IC95% 2,3% a 20,9%; n=43) |
| Fotos com resposta conclusiva | 93,5% (IC95% 89,3% a 96,4%; n=168) |
| Cultura correta, quando responde | 89,2% (IC95% 84,1% a 93,6%; n=157) |

| Indicador | Resultado |
|---|---|
| Doença exata correta, quando responde | 67,5% (IC95% 60,5% a 74,5%; n=157) |

## 5.4 Fora do cenário ideal

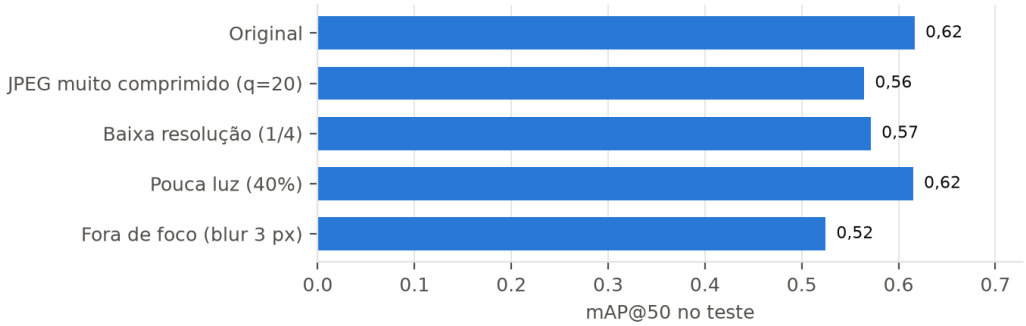



Figura 4. mAP@50 no teste com degradações que simulam câmeras simples e condições ruins de campo.

| Situação | Imagens | Resposta indevida (afirmou doença ou folha sadia) |
|---|---|---|
| Culturas que o modelo não conhece (maçã, pêssego, mirtilo etc.) | 66 | 87,9% (IC95% 80,3% a 95,5%; n=66) |
| Imagens sem planta (COCO128, sem pessoas nem vasos) | 119 | 16,8% (IC95% 10,1% a 23,5%; n=119) |

## 5.5 Tempo real

| Execução (Apple M1, imagem por imagem) | Mediana | p95 | Imagens/s |
|---|---|---|---|
| ONNX CPU, 1 thread(s) | 59,9 ms | 62,6 ms | 16,7 |
| ONNX CPU, 2 thread(s) | 37,6 ms | 41,4 ms | 26,6 |
| ONNX CPU, 4 thread(s) | 27,0 ms | 29,3 ms | 37,0 |

Os tempos incluem pré e pós-processamento. Na demonstração pelo navegador, que inclui rede local e desenho na tela, o sistema manteve uma taxa compatível com uso ao vivo.

## 5.6 Exemplos

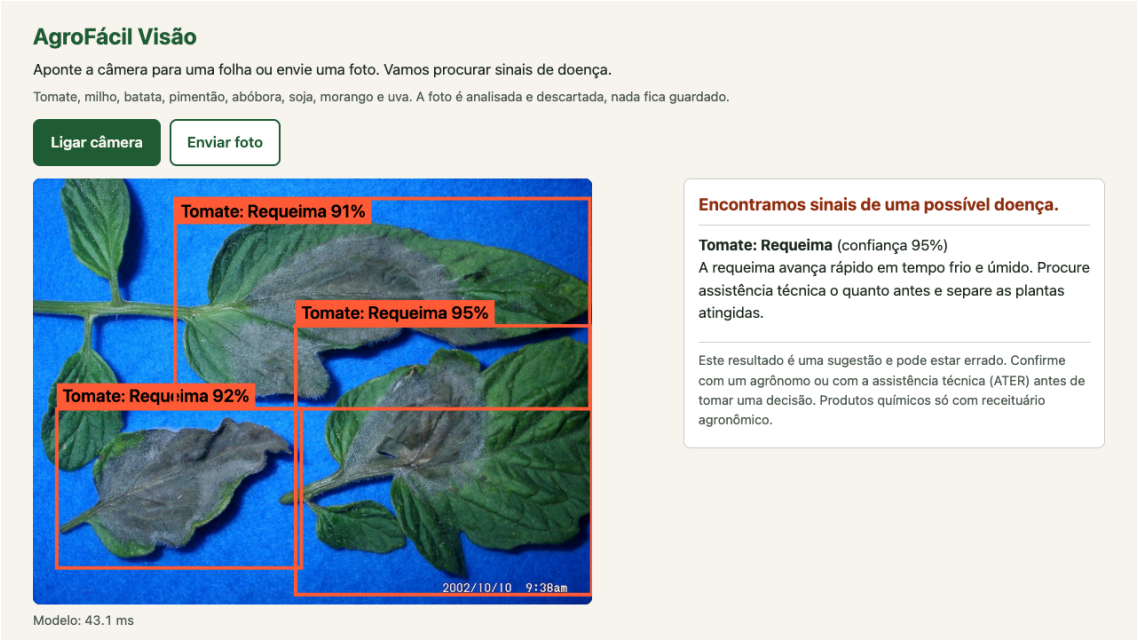



Figura 5. Foto enviada: a caixa indica a folha, e o painel mostra a doença provável, o primeiro cuidado e o aviso de confirmação.




Figura 6. Foto escura e fora de foco: o sistema não arrisca e pede outra foto.




Figura 7. Erro real do modelo no teste: a folha era de uva com podridão-negra, mas o modelo respondeu uva sadia. Por isso a decisão final fica com uma pessoa.

## 6. IA responsável, segurança e LGPD

- As fotos são processadas em memória e descartadas. Os logs registram só status, quantidade de detecções e tempo, sem imagem nem nome de arquivo, o que é verificado por testes automatizados.
- O sistema só afirma algo acima do limiar calibrado. Toda resposta traz o aviso de que é uma sugestão e deve ser confirmada por um agrônomo ou pela ATER. Não recomendamos produto químico nem dose.
- Os limites aceitam no máximo 8 MB e 40 megapixels por imagem, e o serviço responde 503 quando está ocupado, em vez de travar.
- Viés conhecido: as fotos do PlantDoc vêm da internet, com maior presença de Índia e Estados Unidos, e não de lavouras brasileiras.

## 7. Conclusão

Treinamos de forma customizada um detector YOLO11n para 20 classes de folhas e o colocamos em produção em CPU, com cerca de 27 imagens por segundo, integrado ao AgroFácil por uma API que não guarda fotos. No teste, o mAP@50 foi de 0,617, e o sistema sinalizou 89,6% das folhas doentes. O resultado é compatível com o uso como triagem: chamar atenção para a folha certa e incentivar a busca de orientação, nunca substituir o diagnóstico.

A principal lição foi a qualidade dos dados. Duplicatas e vazamento entre treino e teste teriam inflado as métricas sem que percebêssemos. Medir a decisão final por foto e o comportamento fora do cenário ideal mostrou limites que o mAP sozinho esconde.

Próximos passos: coletar e anotar fotos de lavouras brasileiras com produtores e técnicos parceiros; incluir pragas com um dataset próprio; adicionar imagens sem folhas conhecidas ao treino, para reduzir respostas indevidas; e executar o modelo no próprio celular, para funcionar sem internet.

## Referências


SINGH, D. et al. PlantDoc: A Dataset for Visual Plant Disease Detection. CoDS-COMAD 2020. DOI 10.1145/3371158.3371196.

JOCHER, G.; QIU, J. Ultralytics YOLO11. 2024. https://github.com/ultralytics/ultralytics.

ONNX Runtime. https://onnxruntime.ai.

LIN, T.-Y. et al. Microsoft COCO: Common Objects in Context. ECCV 2014.